Brute Force | Pattern Matching & Problem Applications

Lectures 12–13 • Brute-force design technique • pattern matching • applications • exhaustive search

TEXT

B A N A N A S

A N A

shift → compare → match

LECTURE 12

A straightforward design strategy applied to strings

General categories of computational problems

Before choosing an algorithm, first identify what kind of problem is being solved.

Why classify problems?

Problem type often suggests useful data structures and design techniques.

One problem may fit more than one category

For example, shortest path is a graph problem and also an optimization problem.

Algorithms exploit structure

Ordering, graph connectivity, geometry, or optimal-substructure can make a problem much easier.

Problem type → useful structure → suitable algorithmic technique

The categories on the next slides are a practical map, not mutually exclusive boxes.

Problem categories I

Searching • Sorting • String Processing

Searching

Definition: locate a required item or determine whether it exists.

Example: find student ID 1042 in a record set.

Representative algorithms:
• Linear Search
• Binary Search

Sorting

Definition: arrange items according to a key or order.

Example: sort exam scores from highest to lowest.

Representative algorithms:
• Selection Sort
• Merge Sort

String Processing

Definition: search, compare, transform, or analyze sequences of characters.

Example: find “COMSATS” inside a document.

Representative algorithms:
• Naive String Matching
• KMP

Problem categories II

Graph • Combinatorial • Optimization

Graph Problems

Definition: solve problems involving vertices and edges.

Example: find whether two campuses are connected through a network.

Representative algorithms:
• BFS / DFS
• Dijkstra’s Algorithm

Combinatorial Problems

Definition: choose or arrange objects among many possible combinations.

Example: assign 4 tasks to 4 people.

Representative algorithms:
• Exhaustive Enumeration
• Backtracking

Optimization Problems

Definition: find the best feasible solution according to an objective.

Example: choose items that maximize value under a weight limit.

Representative algorithms:
• Greedy Methods
• Dynamic Programming

Problem categories III

Numerical • Geometric • Decision

Numerical Problems

Definition: compute numerical values or solve mathematical systems.

Example: solve a system of linear equations.

Representative algorithms:
• Gaussian Elimination
• Newton’s Method

Geometric Problems

Definition: compute relationships among points, lines, or shapes.

Example: find the closest pair among GPS locations.

Representative algorithms:
• Brute-Force Closest Pair
• Graham Scan

Decision Problems

Definition: answer a yes/no question about an input.

Example: does a path exist from A to B?

Representative algorithms:
• BFS / DFS Reachability
• SAT Backtracking

Summary — computational problem categories

Different problem structures suggest different families of algorithms.

Category	Simple example	Common algorithms
Searching	Find a student record	Linear Search, Binary Search
Sorting	Order marks	Selection Sort, Merge Sort
String Processing	Find a word in text	Naive Matching, KMP
Graph	Find routes / connectivity	BFS, DFS, Dijkstra
Combinatorial	Arrange or choose objects	Enumeration, Backtracking
Optimization	Find best feasible solution	Greedy, Dynamic Programming
Numerical	Solve equations	Gaussian Elimination, Newton
Geometric	Closest points / hull	Closest Pair, Graham Scan
Decision	Yes/no property	Reachability, SAT Search

When and why do we use brute force?

Brute force is appropriate when its simplicity and completeness outweigh its extra work.

Small search space

The input or candidate set is small enough that checking everything is practical.

No obvious exploitable structure

There may be no ordering, recurrence, graph property, or other structure that immediately helps.

Hard to reduce or prune

We may not know how to safely discard large parts of the search space.

No better practical approach

A more sophisticated method may be unavailable, unnecessary, or too costly to develop.

Simplicity matters

Direct algorithms are often easier to implement, test, explain, and maintain.

Two especially important uses of brute force

Sometimes exhaustive examination is exactly what we need.

Guaranteeing an optimal answer

If every feasible candidate is evaluated, the best candidate cannot be missed.

Example: exhaustive TSP on a very small number of cities.

Creating a verification baseline

A simple exact method can act as an “oracle” for small inputs. Compare an optimized algorithm against it to catch bugs.

Small exact baseline → test sophisticated algorithm → confidence in correctness

Important nuance: “cannot divide” does not mean “must brute-force”

Algorithm selection is broader than Divide-and-Conquer versus Brute Force.

Maybe transform the problem

Change the representation or preprocess the data.

Example: sort once, then use Binary Search.

Maybe use another design technique

Greedy, Dynamic Programming, Decrease-and-Conquer, graph algorithms, hashing, and others may still apply.

Maybe exploit a domain property

Constraints, ordering, sparsity, monotonicity, geometry, or repeated queries can change the best approach.

Brute force is one tool in the algorithmic toolbox — not the automatic fallback whenever division is difficult.

Brute force across different problem categories

The same design philosophy can produce very different running times.

Problem	Brute-force idea	Candidate count / work	Complexity
Linear Search	Inspect items one by one	At most n items	O(n)
Pair with target sum	Try every unordered pair	$n(n-1)/2$ pairs	$O(n^2)$
Naive String Matching	Try pattern at every legal shift	$(n-m+1)$ shifts × up to m comparisons	O(nm)
Traveling Salesperson	Evaluate every possible tour	Factorially many tours	O(n!)

The algorithmic idea is “systematically try the natural candidates.” The complexity depends on how many candidates exist and how expensive each test is.

Brute force is a design technique — not a complexity class

Do not attach one Big-O value to the words “brute force.”

Linear Search → O(n)

Target-Sum Pair → $O(n^2)$

Naive String Matching → O(nm)

Exhaustive TSP → O(n!)

Same design philosophy — very different growth rates.

Therefore, analyze the candidate space for the specific problem instead of saying “brute force is $O(n^2)$” or “brute force is exponential.”

Why use brute force at all?

Because “simple and direct” is often a feature, not a failure.

Start with a correct baseline

The most direct solution is usually easy to explain, implement, and verify.

Solve small instances quickly

For small n, a theoretically slower method may finish fast enough that extra sophistication is unnecessary.

Use it as a reference

A brute-force answer is useful for testing optimized algorithms and catching mistakes.

No preprocessing needed

Often you can begin immediately with the raw input.

Sometimes it is the only obvious method

When no better structure is known, systematic enumeration still guarantees an exact answer.

Brute force gives us a dependable starting point. Optimization comes after we understand the baseline.

When is brute force a good choice?

Think about input size, how often the problem is solved, and how expensive mistakes would be.

Small input

The candidate space is tiny enough to inspect directly.

One-off task

Building preprocessing structures would cost more than the search itself.

Need an exact answer

Systematic checking does not rely on heuristics or approximation.

Prototype or teaching

The logic is transparent, so behavior and complexity are easy to trace.

Verification oracle

Compare an optimized implementation against brute-force answers on small cases.

Rare operation

A simple $O(n^2)$ routine can be perfectly acceptable when it runs only occasionally.

When does brute force become a bad choice?

The same simplicity can become expensive when the candidate space grows quickly.

Large n

Quadratic, cubic, exponential, and factorial candidate spaces can become impractical very quickly.

Repeated queries

If the same data is searched many times, preprocessing or a better data structure can pay off.

Real-time constraints

A direct exhaustive search may be too slow when a result is needed immediately.

n^2 → manageable for moderate n | 2^n or n! → explosive growth

The question is not “Is brute force bad?” The question is “Is the candidate space small enough for this workload?”

Brute force vs. exhaustive search

They are related, but the terms are not exactly identical.

Brute force

A broad design idea: use the most direct, straightforward method suggested by the problem.

Examples: sequential search, selection sort, naive string matching.

Exhaustive search

A specific brute-force pattern: generate every candidate solution, test each one, and keep the best or valid one.

Examples: TSP tours, knapsack subsets, assignment permutations.

Every exhaustive-search algorithm is brute-force in spirit, but not every brute-force algorithm enumerates complete solutions.

The general brute-force pattern

Before coding, identify what counts as a candidate and how expensive it is to test.

1

Generate

Choose the next natural candidate.

2

Test

Check whether the candidate satisfies the required condition.

3

Keep / count

Store the best, record valid candidates, or update a count.

4

Repeat

Continue until every required candidate has been examined.

Total work ≈ number of candidates × cost of testing one candidate

What should you be able to do by the end?

Understand the algorithm first, then trace and analyze its cost.

DEFINE

State the text–pattern matching problem precisely.

TRACE

Move the pattern across the text and show comparisons.

COUNT

Count character comparisons for a concrete run.

ANALYZE

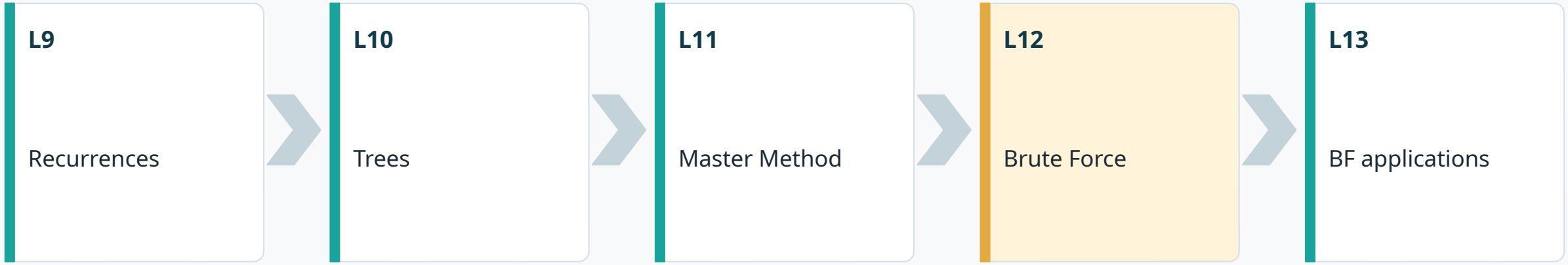
Derive best- and worst-case running time.

JUDGE

Explain when brute force is reasonable and when it scales poorly.

A new phase of the course

We are moving from “how do I solve a recurrence?” to “how do I design an algorithm?”



Analysis tools tell us how much work an algorithm performs. Design techniques tell us how to build candidate algorithms in the first place.

What does “brute force” mean?

Start with the most direct interpretation of the problem and try the obvious possibilities systematically.

DIRECT

Use the problem statement almost literally.

For pattern matching: try the pattern at each possible starting position.

SYSTEMATIC

Do not guess random positions.

Move left to right and compare characters in a fixed order.

SIMPLE

Usually easy to understand, implement, and verify.

The tradeoff: it may repeat work.

Define the pattern-matching problem

We need a precise input and output before discussing the algorithm.

TEXT T

A sequence of n characters.
Example: BANANAS

n = 7

PATTERN P

A shorter sequence of m characters.
Example: ANA

m = 3

OUTPUT

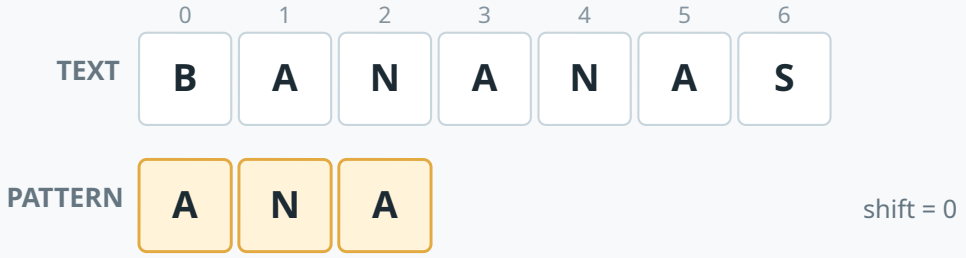
Find an index s such that
T[s ... s+m−1] = P.

If no such s exists, report NOT FOUND.

Assume 0-based indexing in this lecture.

How many starting positions are possible?

A length-m pattern must fit completely inside a length-n text.



Valid starts: 0, 1, 2, …, n−m

Number of alignments = (n−m)+1 = n−m+1

For n=7 and m=3: 7−3+1 = 5 possible alignments. If m>n, there is no legal alignment.

The brute-force algorithm in one sentence

At each alignment, compare pattern characters left to right until a mismatch or a complete match occurs.

1 — ALIGN

Place P at shift s.

2 — COMPARE

Check P[0], P[1], …

3 — STOP EARLY

On mismatch, move to s+1.

4 — REPORT

If all m match, return s.

Pseudocode

The structure is two nested scans, but the inner scan may stop early.

PSEUDOCODE

```
BRUTE-FORCE-MATCH(T, P)
   n ← length(T)
   m ← length(P)

   for s ← 0 to n − m
      j ← 0
      while j < m and T[s + j] = P[j]
         j ← j + 1

      if j = m
         return s

   return NOT FOUND
```

OUTER LOOP

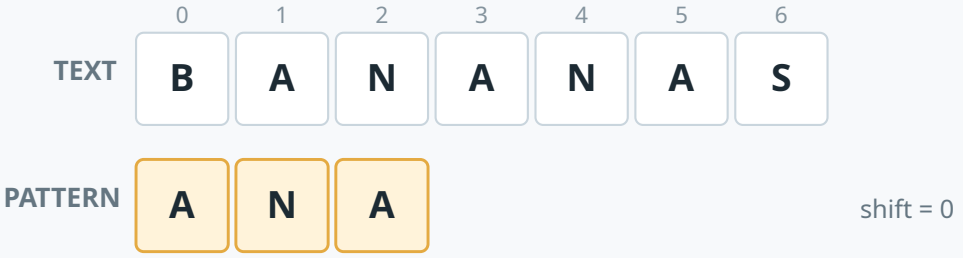
Tries each legal alignment s = 0 … n−m.

INNER LOOP

Compares characters until mismatch or all m match.

Worked example 1: find ANA in BANANAS

Use the same text for several slides and count every comparison.



QUESTION

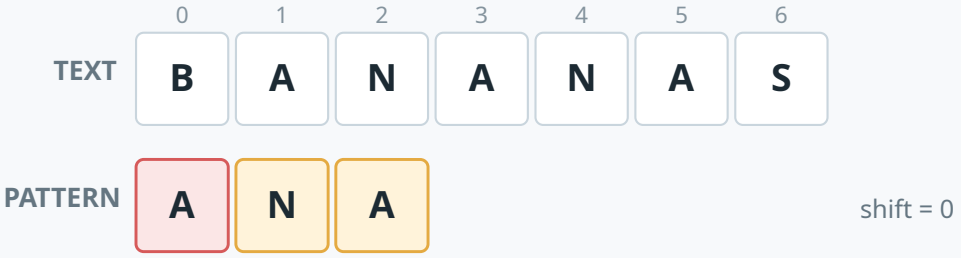
At which shift will the first full match occur?

COUNTER

We will maintain C = number of character comparisons.

Shift 0: fail immediately

Compare T[0] with P[0].



COMPARISON 1

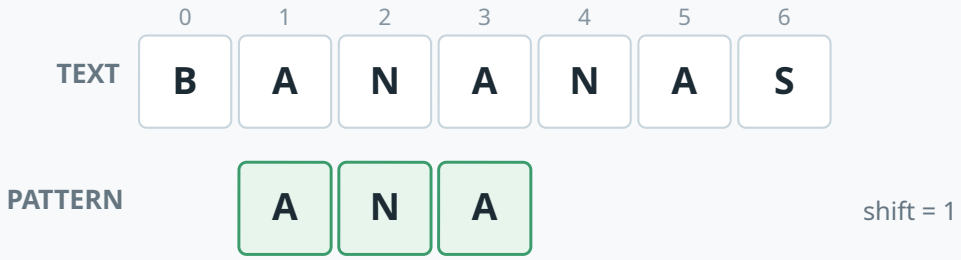
B ≠ A
Mismatch on the first character.

ACTION

Stop this alignment immediately.
Move pattern one position right.

Shift 1: compare left to right

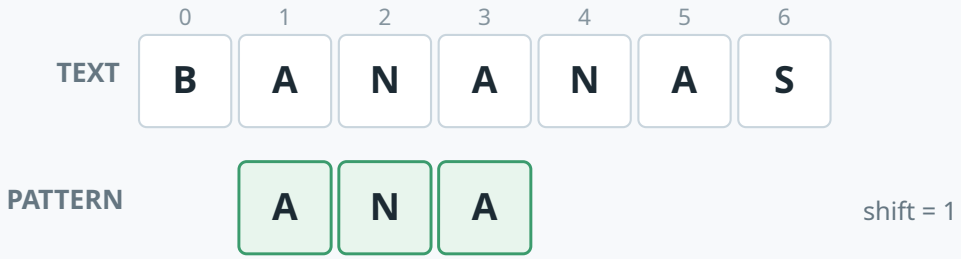
Now the first two characters match — keep going.



Comparison #	Text char	Pattern char	Result
2	T[1]=A	P[0]=A	match
3	T[2]=N	P[1]=N	match
4	T[3]=A	P[2]=A	match

A full match is found at shift 1

All m=3 pattern characters matched.



Return index 1

Total character comparisons = 1 + 3 = 4

Important: mismatches can save work

The inner comparison loop does not always execute m times.

EARLY MISMATCH

If the first character differs, only 1 comparison is used for that alignment.

Example: B vs A at shift 0.

LATE MISMATCH

If the first m−1 characters match and the last differs, almost the whole pattern is checked.

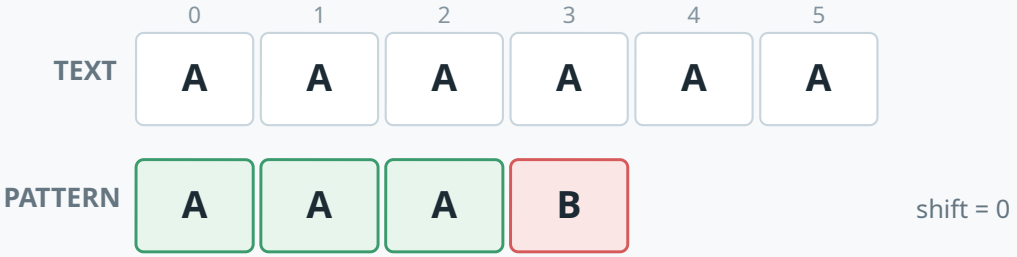
This is expensive.

FULL MATCH

A successful alignment requires m comparisons because every pattern character must agree.

Worked example 2: a deliberately expensive failure

Text = AAAAAA, Pattern = AAAB

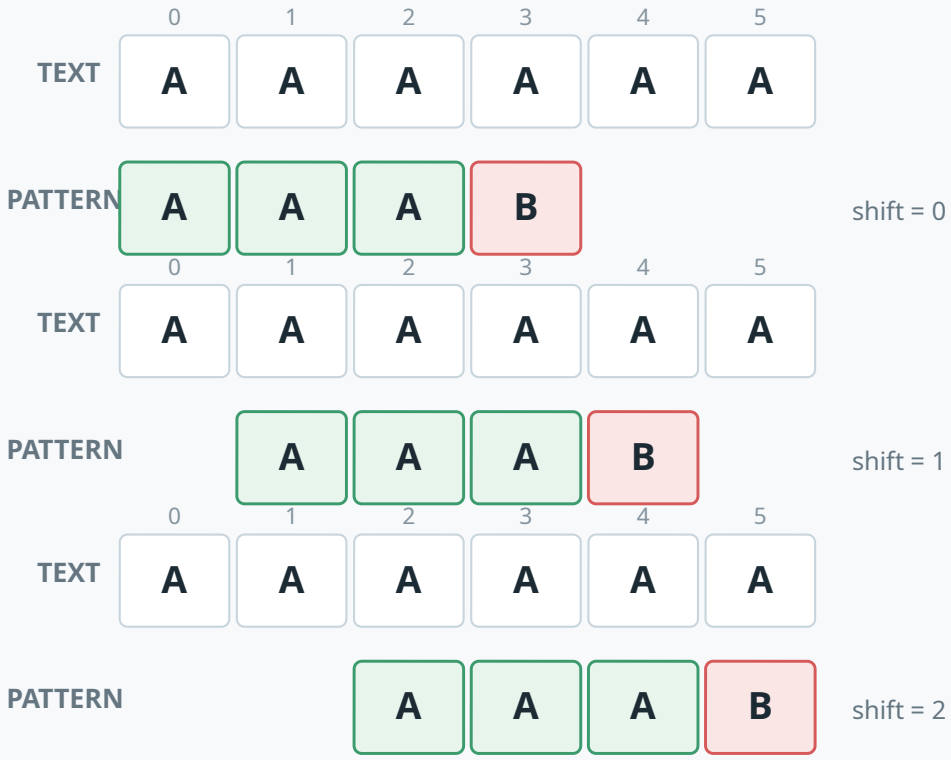


At shift 0: A=A, A=A, A=A, A≠B → 4 comparisons

Because the mismatch comes last, this alignment uses the maximum m comparisons.

The same late mismatch repeats at every alignment

n=6, m=4, so there are n−m+1 = 3 alignments.



3 alignments × 4 comparisons each = 12 comparisons

Derive the worst-case comparison count

Worst case means every legal alignment uses all m comparisons.

ALIGNMENTS

$n-m+1$

COMPARISONS PER ALIGNMENT

m

TOTAL

$m(n-m+1)$

$$C_{worst}(n,m) = m(n-m+1)$$

From the exact count to asymptotic growth

Now simplify the expression without losing the scaling behavior.

$$C_{worst}(n,m) = m(n-m+1) \leq mn$$

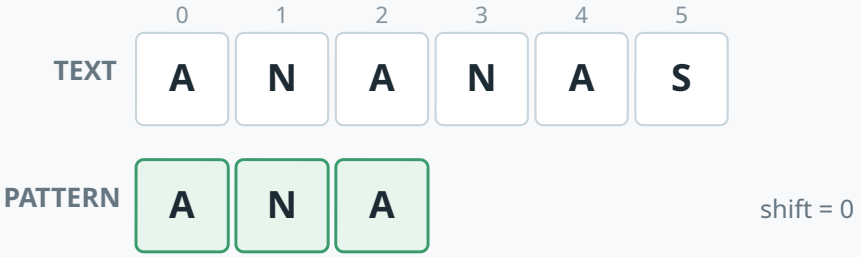
Therefore worst-case time = O(nm)

When m is proportional to n, this can become quadratic: $\Theta(n^2)$.

When m is a small fixed constant, the scan behaves approximately linearly in n.

Best case: match immediately at the first alignment

To confirm a match, the algorithm must compare all m pattern characters.



m comparisons → Θ(m)

For the search problem, one early mismatch does not finish the whole algorithm — it only rejects one alignment.

Best, typical-looking, and worst behavior

The same algorithm can perform very different numbers of character comparisons.

Case	What happens	Comparison count	Growth
Best	Pattern matches at shift 0	m	$\Theta(m)$
Early mismatches	Most alignments fail quickly	often far below $m(n-m+1)$	input-dependent
Worst	Each alignment mismatches very late	$m(n-m+1)$	$\Theta(m(n-m+1))$

The worst-case bound is a guarantee; it does not claim every input takes that much work.

What does O(nm) look like numerically?

A modest pattern can still cause many comparisons on a large text.

n (text)	m (pattern)	alignments	worst comparisons
1,000	10	991	9,910
1,000,000	20	999,981	19,999,620
1,000,000	500	999,501	499,750,500

The algorithm is simple; repeated re-checking is what can become expensive.

Where does the wasted work come from?

Brute force forgets what it learned at the previous alignment.



The second alignment re-compares many A characters that look almost identical to the previous work.

More advanced string-matching algorithms use information from earlier comparisons to skip some of this repetition.

Space complexity

The straightforward implementation needs very little extra memory.

STORED INPUT

Text T and pattern P are already given as input.

EXTRA STATE

Only a few indices/counters such as s and j are required.

AUXILIARY SPACE

Θ(1) extra space in the usual iterative implementation.

Why brute force is still useful

A slower asymptotic method can still be the sensible engineering choice.

No preprocessing

Start searching immediately.

Tiny implementation

Easy to code and test.

Low memory

Constant extra space.

Good baseline

Useful for checking a more complex algorithm.

Small inputs

Overhead of advanced methods may not be worth it.

When does brute force become unattractive?

Repeated comparisons accumulate when both text and pattern are large or highly repetitive.

LARGE TEXTS

Scanning many candidate positions can be expensive.

LONG PATTERNS

Each alignment may require many character checks.

REPETITIVE DATA

Late mismatches can force the algorithm to redo nearly the same work.

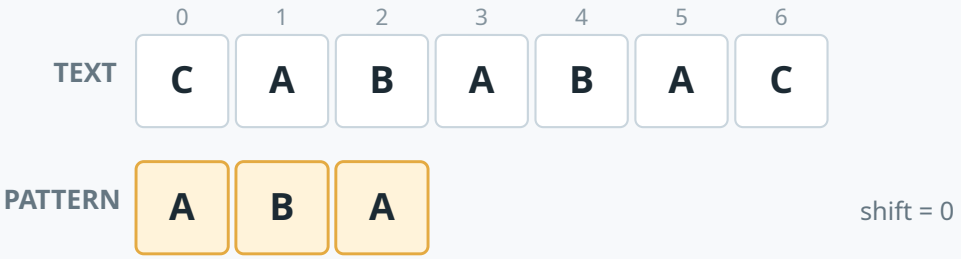
Technique selection perspective

The “best” algorithm depends on the workload, not only on the asymptotic label.

Situation	Likely choice	Reason
Short text + one search	Brute force	Simple; almost no setup cost
Large text + one short pattern	Brute force may still be fine	m is small
Huge text + long/repetitive pattern	Consider a smarter matcher	Repeated work can dominate
Many searches on stable data	Preprocessing/indexing may help	Setup cost can be reused

Practice 1: trace the search

Text = CABABAC, Pattern = ABA



Your tasks

1. List each shift examined.
2. Count comparisons at each shift.
3. Give the returned index.
4. Give the total number of comparisons.

Practice 1 answer

Work alignment by alignment rather than jumping to the final index.

Shift	Comparisons	Count	Result
0	C vs A	1	mismatch
1	A=A, B=B, A=A	3	match → return 1

Total comparisons = 1 + 3 = 4

Practice 2: derive the worst-case count

Suppose n=20 and m=5.

ALIGNMENTS

$n-m+1 = 20-5+1 = 16$

MAX PER ALIGNMENT

$m = 5$ comparisons

WORST TOTAL

$16 \times 5 = 80$ comparisons

General formula: $m(n-m+1)$

Practice 3: which input is more expensive?

Both use the same n and m — arrangement still matters.

A — MANY EARLY MISMATCHES

Text: ZXQWERTYUIOP
Pattern: AAAA

Most alignments fail at the first character.

B — MANY LATE MISMATCHES

Text: AAAAAAAAAAAA
Pattern: AAAB

Many alignments match three characters before failing.

B is typically much more expensive even though n and m are identical.

Common mistakes

Most errors come from counting or indexing, not from difficult mathematics.

MISTAKE 1

Using $n-m$ alignments instead of $n-m+1$.

MISTAKE 2

Continuing comparisons after a mismatch.

MISTAKE 3

Forgetting that a successful match needs all m comparisons.

MISTAKE 4

Calling worst-case $O(n^2)$ for every possible m.

MISTAKE 5

Ignoring input arrangement when discussing actual comparisons.

A repeatable solution method

For a trace or analysis question, use the same five-step routine.

1. Write n and m.
2. Compute n−m+1 candidate shifts.
3. Trace comparisons at each shift.
4. Count comparisons explicitly.
5. State best/worst growth and explain why.

Final summary

Brute force is simple enough to understand completely — which makes it a perfect first design technique.

ALGORITHM

Try every legal alignment.
Compare left to right.
Stop an alignment on mismatch.
Return on full match.

ANALYSIS

Alignments: $n-m+1$
Worst comparisons: $m(n-m+1)$
Worst time: $O(nm)$
Extra space: $\Theta(1)$

JUDGMENT

Excellent for simplicity and small inputs.
Can repeat work badly on large/repetitive inputs.
Next: other brute-force applications.

Reading and practice

Use the worked examples to reproduce the traces by hand.

TEXTBOOK

A. Levitin, Introduction to the Design and Analysis of Algorithms
Brute-force design technique and string matching.

REFERENCE

T. H. Cormen et al., Introduction to Algorithms
Use the analysis tools from earlier lectures when counting work.

Recommended practice: trace at least one successful match, one failure with early mismatches, and one late-mismatch worst-style input.

Brute Force | Problem Applications

Same idea.
Different problems.

Part II • Inversion counting • closest pair • convex hull

INVERSIONS

compare pairs

1 4 2 3

CLOSEST PAIR

compare distances

• • •

CONVEX HULL

test boundary edges

△ • ◇

What should you be able to do by the end?

Apply the brute-force design idea to three different problem types.

INVERSION	Identify and count out-of-order pairs.
PAIR	Compare point pairs and find the minimum distance.
HULL	Decide whether a line segment can be a hull edge.
COUNT	Derive the number of candidate pairs tested.
JUDGE	Explain why brute force is useful — and where it becomes expensive.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

One design idea — three very different problems

Brute force means systematically testing all natural candidates.

Inversion count

candidate = pair of array positions

TEST

check whether order is reversed

Closest pair

candidate = pair of points

TEST

compute distance and keep minimum

Convex hull

candidate = pair of points as an edge

TEST

check whether all other points lie on one side

The recurring brute-force pattern: generate every reasonable candidate → test it → keep/count the valid ones.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Application 1 — Inversion Counting

An inversion measures how far an array is from being sorted.

For positions i < j, the pair (A[i], A[j]) is an inversion when

A[i] > A[j]

Earlier element **Later element**

Example



(2,1), (4,1), (4,3) are inversions.

Sorted array → 0 inversions.
Reverse-sorted array → maximum inversions.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Brute-force inversion count

Compare every pair of positions i < j.

```
count = 0
for i = 0 to n−2
    for j = i+1 to n−1
        if A[i] > A[j]
            count = count + 1
return count
```

WHY i < j?

Each unordered position pair should be tested exactly once.

WHAT DO WE COUNT?

One comparison A[i] > A[j] for each candidate pair.

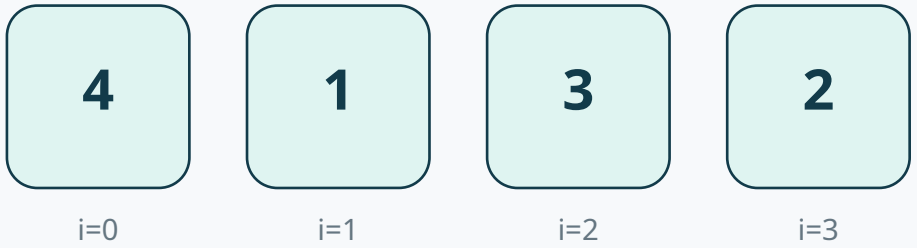
OUTPUT

The total number of inversions.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Worked example — count inversions in [4, 1, 3, 2]

We will enumerate every pair once.



There are C(4,2)=6 candidate pairs.

We test:

(4,1) **(4,3)** **(4,2)**

(1,3) **(1,2)** **(3,2)**

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Trace — fix i = 0

A[0]=4 is compared with every element to its right.

4 > 1	**inversion?**	**YES**	**count = 1**
4 > 3	**inversion?**	**YES**	**count = 2**
4 > 2	**inversion?**	**YES**	**count = 3**

After finishing i=0, we have already found 3 inversions.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Complete inversion trace

Continue with i=1 and i=2.

pair	comparison	inversion?	running count
i=0, j=1	4 > 1	yes	1
i=0, j=2	4 > 3	yes	2
i=0, j=3	4 > 2	yes	3
i=1, j=2	1 > 3	no	3
i=1, j=3	1 > 2	no	3
i=2, j=3	3 > 2	yes	4

Final answer: 4 inversions

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

How many pairs does brute force test?

The nested loops examine every pair i < j exactly once.

For i=0: n−1 comparisons

For i=1: n−2 comparisons

For i=2: n−3 comparisons

...

Last useful i: 1 comparison

Total comparisons

$$(n-1)+(n-2)+\cdots+2+1$$

$$= n(n-1)/2$$

$$= \Theta(n^2)$$

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Minimum and maximum inversion counts

Running time is still $\Theta(n^2)$, even though the number of inversions varies.

Sorted

1	2	3	4	5

0 inversions

Reverse sorted

5	4	3	2	1

$n(n-1)/2$ inversions

The brute-force algorithm still checks all pairs in both cases.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

A preview — can inversion counting be faster?

Yes. Later, divide and conquer can count inversions during merging.

BRUTE FORCE

Compare every pair.

Candidate pairs = $n(n-1)/2$

Time = $\Theta(n^2)$

DIVIDE & CONQUER PREVIEW

Split the array, count inside each half, then count cross-inversions while merging.

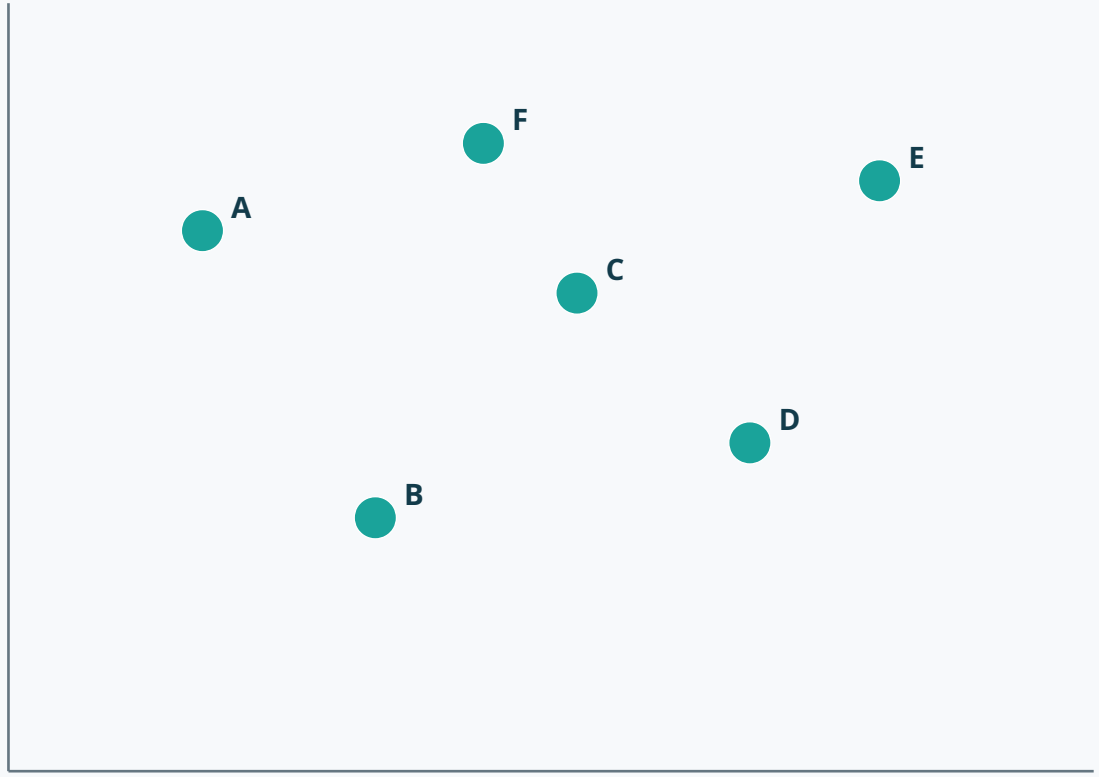
Time = $\Theta(n \log n)$

Same problem. Different design technique. Different asymptotic cost.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Application 2 — Closest Pair of Points

Given n points in the plane, find the pair with minimum Euclidean distance.



Distance formula

$$d(P_i,P_j) = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$$

Brute force: compute this for every pair of points.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Brute-force closest-pair algorithm

Generate every pair, compute its distance, keep the smallest.

```
best = ∞
for i = 0 to n−2
    for j = i+1 to n−1
        d = distance(P[i], P[j])
        if d < best
            best = d
            pair = (P[i], P[j])
return pair, best
```

CANDIDATES

Every unordered pair of points.

TEST

Compute Euclidean distance.

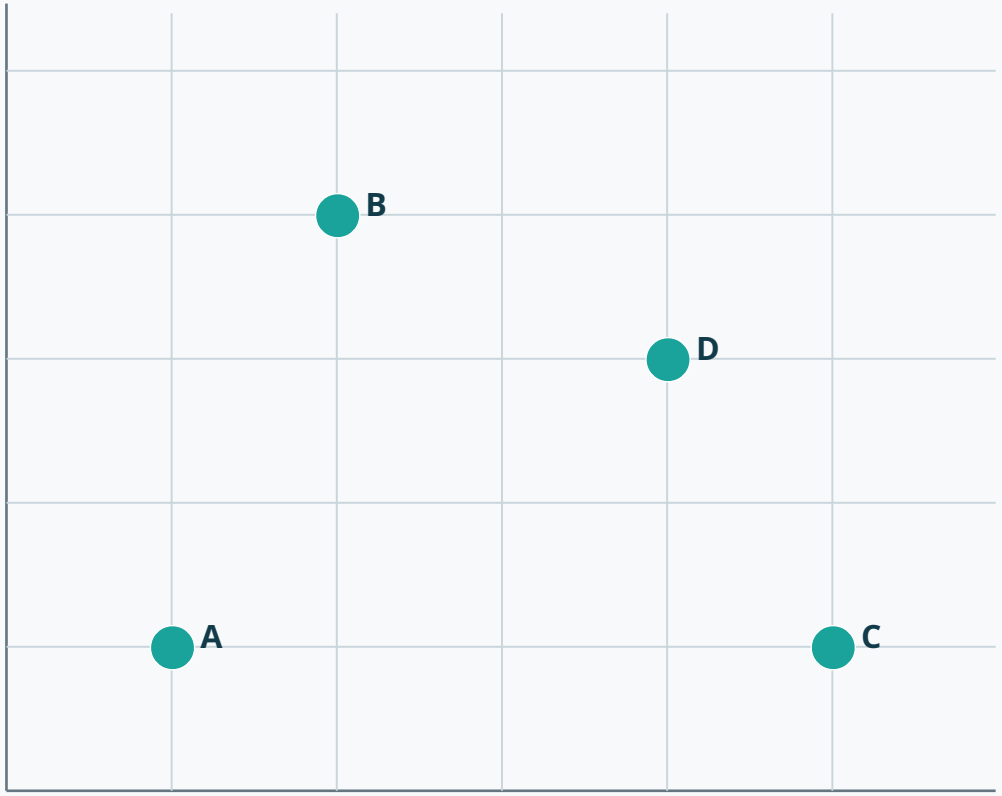
KEEP

Smallest di

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Worked example — four points

Let A=(1,1), B=(2,4), C=(5,1), D=(4,3).



Candidate pairs

AB	AC
AD	BC
BD	CD

Again: C(4,2)=6 pairs.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Distance calculations — first three pairs

Do not skip the arithmetic: square, add, then square-root.

AB	$\sqrt{((1-2)^2+(1-4)^2)}$	$\sqrt{(1+9)}=\sqrt{10}$	**≈ 3.162**
AC	$\sqrt{((1-5)^2+(1-1)^2)}$	$\sqrt{(16+0)}=\sqrt{16}$	**$= 4$**
AD	$\sqrt{((1-4)^2+(1-3)^2)}$	$\sqrt{(9+4)}=\sqrt{13}$	**≈ 3.606**

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Distance calculations — remaining pairs

Continue and update the current minimum.

BC	$\sqrt{((2-5)^2+(4-1)^2)}$	$\sqrt{(9+9)}=\sqrt{18}$	**≈ 4.243**
BD	$\sqrt{((2-4)^2+(4-3)^2)}$	$\sqrt{(4+1)}=\sqrt{5}$	**≈ 2.236**
CD	$\sqrt{((5-4)^2+(1-3)^2)}$	$\sqrt{(1+4)}=\sqrt{5}$	**≈ 2.236**

Closest distance = $\sqrt{5} \approx 2.236$; pairs BD and CD tie.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

How the running minimum changes

The algorithm needs only the best value seen so far.

pair tested	distance	best so far
AB	3.162	**3.162**
AC	4.000	**3.162**
AD	3.606	**3.162**
BC	4.243	**3.162**
BD	2.236	**2.236**
CD	2.236	**2.236**

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Closest-pair complexity

The expensive part is not the distance formula — it is how many pairs we test.

Number of point pairs

$$C(n,2)$$

$$= n(n-1)/2$$

$$= \Theta(n^2)$$

TIME

$\Theta(n^2)$ pair checks; each distance calculation is constant-time.

EXTRA SPACE

$\Theta(1)$ if we keep only the current best pair.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

When is brute-force closest pair reasonable?

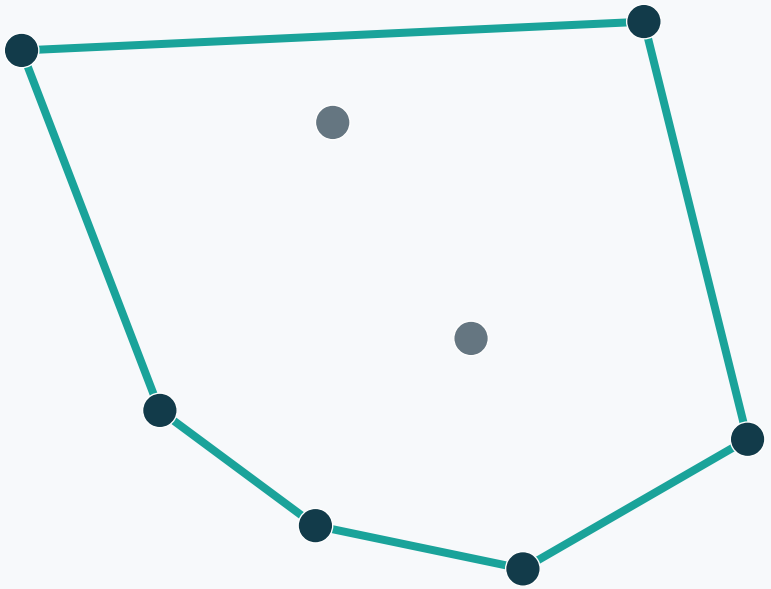
Simple algorithms can still be the right engineering choice.

GOOD	Small n; one-off analysis; easy implementation; correctness is obvious.
BAD	Large point sets; repeated queries; performance-sensitive geometry.
LESSON	As n grows, testing all $n(n-1)/2$ pairs becomes the bottleneck.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Application 3 — Convex Hull

The convex hull is the smallest convex boundary containing all points.



Boundary points

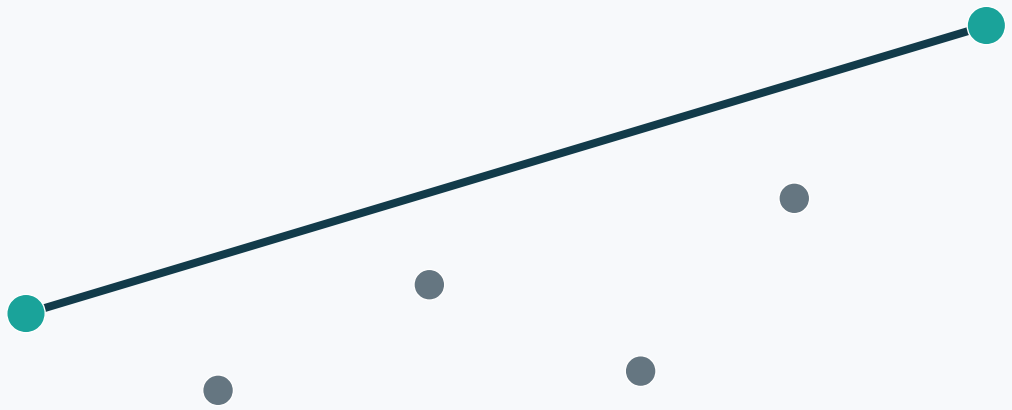
Vertices of the convex hull form a polygon that contains every input point.

Interior points are not hull vertices.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Brute-force idea for a hull edge

Pick two points P_i and P_j. Could the segment P_iP_j lie on the boundary?



If every other point lies on the same side of the directed line $P_i \rightarrow P_j$, the segment is a hull-edge candidate.

If points appear on both sides, the segment passes through the interior and cannot be a hull edge.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

How do we tell which side a point is on?

Use the orientation / signed-area expression.

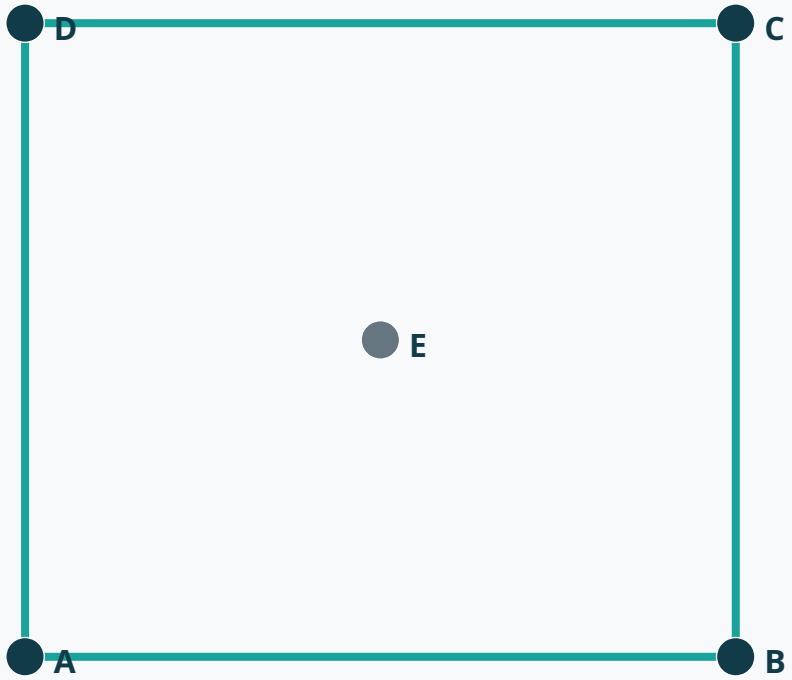
$$\text{value} = (x_j - x_i)(y_k - y_i) - (y_j - y_i)(x_k - x_i)$$

value > 0	P_k lies on one side of directed line $P_i \rightarrow P_j$
value < 0	P_k lies on the opposite side
value = 0	The three points are collinear

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Worked hull example — five points

A=(0,0), B=(4,0), C=(4,4), D=(0,4), E=(2,2).



Expected hull

A → B → C → D → A

E is inside the square, so it is not a hull vertex.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Test candidate edge AB

Check the orientation of every other point relative to directed line A→B.

test point	orientation expression	value	sign
C=(4,4)	(4−0)(4−0)−(0−0)(4−0)	**16**	**+**
D=(0,4)	(4−0)(4−0)−(0−0)(0−0)	**16**	**+**
E=(2,2)	(4−0)(2−0)−(0−0)(2−0)	**8**	**+**

All signs are positive → all other points are on the same side → AB is a hull edge.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Test a bad candidate — AE

A segment through the interior should fail because points lie on both sides.

B=(4,0)	$(2)(0)-(2)(4)$	**-8**	**negative**
D=(0,4)	$(2)(4)-(2)(0)$	**$+8$**	**positive**

We already have opposite signs. Stop: AE cannot be a hull edge.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Brute-force convex-hull outline

For each candidate pair, verify that every remaining point stays on one side.

```
for each pair (P[i], P[j])
    side = 0
    valid = true
    for each k ≠ i,j
        s = orientation(P[i],P[j],P[k])
        if s has opposite sign
            valid = false; break
    if valid: record edge (i,j)
```

OUTER CHOICE

Choose a candidate pair: $\Theta(n^2)$ possibilities.

INNER TEST

Check up to $n-2$ other points for that pair.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Convex-hull brute-force complexity

Pair candidates times point checks.

candidate pairs

$\Theta(n^2)$

×

points checked per pair

$\Theta(n)$

=

total

$\Theta(n^3)$

This is why brute-force convex hull becomes expensive much sooner than the other two examples.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Compare the three brute-force applications

The candidate space determines the running time.

problem	candidate	test per candidate	count	time
Inversions	pair of positions	compare values	$\Theta(n^2)$	$\Theta(n^2)$
Closest pair	pair of points	compute distance	$\Theta(n^2)$	$\Theta(n^2)$
Convex hull	pair as edge	scan all points	$\Theta(n^2)$	$\Theta(n^3)$

Brute force is not one fixed complexity. The cost depends on how many candidates exist and how expensive each test is.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

A general brute-force analysis pattern

Use the same four questions on new problems.

1. **What are the candidates?**
2. **How many candidates are there?**
3. **How much does one candidate test cost?**
4. **Multiply — then simplify asymptotically.**

This is the bridge from “design the brute-force algorithm” to “analyze its cost.”

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Practice 1 — inversion count

Array: [5, 2, 4, 1, 3]

A. List every inversion.

B. How many candidate pairs are tested?

C. What is the brute-force time complexity?



Work it manually before looking at the answer slide.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Practice 2 — closest pair

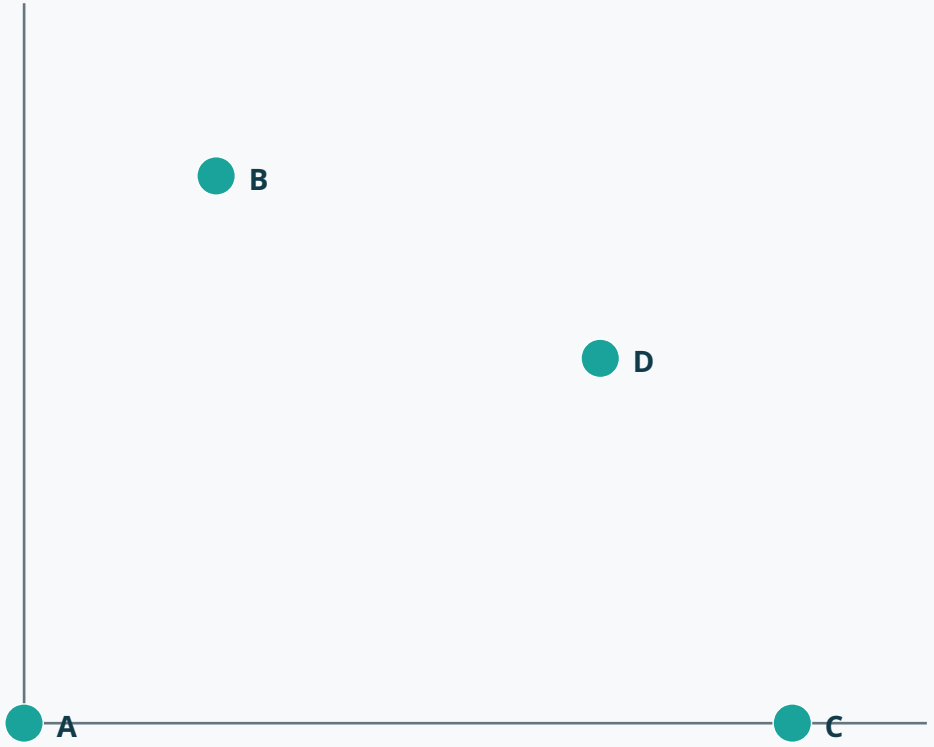
Points: A=(0,0), B=(1,3), C=(4,0), D=(3,2)

A. Enumerate all 6 pairs.

B. Compute each distance.

C. Identify the closest pair.

D. State the brute-force complexity.



CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Practice 3 — hull-edge test

Use A=(0,0), B=(5,0), C=(4,3), D=(1,4), E=(2,2).

Test whether AB is a convex-hull edge.

Compute orientation(A,B,C)

Compute orientation(A,B,D)

Compute orientation(A,B,E)

Are the signs consistent?

Decision rule

same side for all tested points
→ candidate hull edge

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Practice answers — inversion count

[5, 2, 4, 1, 3]

Inversions:

- (5, 2)
- (5, 4)
- (5, 1)
- (5, 3)
- (2, 1)
- (4, 1)
- (4, 3)

Total inversions = 7

Candidate pairs = C(5,2)=10

Brute-force time = $\Theta(n^2)$

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Practice answers — closest pair

A=(0,0), B=(1,3), C=(4,0), D=(3,2)

AB	√10 ≈ 3.162
AC	4
AD	√13 ≈ 3.606
BC	√10 ≈ 3.162
BD	√5 ≈ 2.236
CD	√5 ≈ 2.236

Closest distance

√5 ≈ 2.236

pairs BD and CD

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Practice answer — hull-edge test

Directed line A→B where A=(0,0), B=(5,0).

C=(4,3)	(5)(3)−(0)(4)=15	+
D=(1,4)	(5)(4)−(0)(1)=20	+
E=(2,2)	(5)(2)−(0)(2)=10	+

All signs are positive → AB is a hull-edge candidate.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Common mistakes to avoid

Most errors come from incomplete candidate generation or incorrect counting.

- Inversions: comparing only adjacent elements instead of every pair $i<j$.
- Closest pair: forgetting a pair or testing both AB and BA.
- Closest pair: comparing squared distances inconsistently with ordinary distances.
- Convex hull: checking only a few points instead of every other point.
- Convex hull: treating mixed positive and negative orientation signs as valid.
- Complexity: assuming every brute-force algorithm is automatically $\Theta(n^2)$.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

Technique-selection perspective

Brute force gives us a clear baseline to improve later.

START SIMPLE A brute-force solution is often the easiest correct baseline.

MEASURE Count candidates and test cost before deciding whether optimization is necessary.

IMPROVE Later techniques can avoid testing every candidate or make each test cheaper.

Next: Divide and Conquer — Merge Sort and Quick Sort.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus

More Brute-Force Examples

Selection Sort • Bubble Sort • Sequential Search • Exhaustive Search

TSP • 0/1 Knapsack • Assignment Problem • Graph-traversal preview

The same design philosophy appears in several classic textbook examples.

LECTURES 12–13

Textbook map: where brute force appears

Levitin’s brute-force chapter uses several different problem types to show the same design philosophy.

Sorting

Selection Sort
Bubble Sort

Searching / strings

Sequential Search
Brute-force String Matching

Geometry

Closest Pair
Convex Hull

Exhaustive search

Traveling Salesman
0/1 Knapsack
Assignment Problem

Graphs

DFS and BFS

We already study pattern matching, closest pair, convex hull, and inversion counting in detail; the following slides add the other classic examples.

Selection Sort as a brute-force sorting method

Repeatedly scan the unsorted suffix and select its smallest element.

Direct idea

Do not build a sophisticated structure. Simply look through everything that remains and choose the minimum.

Candidate

Every unsorted element is a candidate for the next output position.

Action

After finding the minimum, swap it into the first unsorted position.

[5, 2, 4, 1, 3] → choose the minimum from the unsorted part

Selection Sort — worked example

Array = [5, 2, 4, 1, 3]

Pass 1	scan 5,2,4,1,3 → min=1	**[1, 2, 4, 5, 3]**	4 comparisons
Pass 2	scan 2,4,5,3 → min=2	**[1, 2, 4, 5, 3]**	3 comparisons
Pass 3	scan 4,5,3 → min=3	**[1, 2, 3, 5, 4]**	2 comparisons
Pass 4	scan 5,4 → min=4	**[1, 2, 3, 4, 5]**	1 comparison

Total comparisons = 4 + 3 + 2 + 1 = 10

Selection Sort — complexity

The number of comparisons does not depend on the original ordering.

$$C(n) = (n-1) + (n-2) + \cdots + 1$$

$$C(n) = n(n-1)/2$$

$$T(n) = \Theta(n^2)$$

Selection Sort is simple and predictable, but it keeps scanning the remaining elements even when the array is nearly sorted.

Bubble Sort as a brute-force sorting method

Repeatedly compare adjacent pairs and swap them when they are out of order.

Local test

Compare neighbors A[j] and A[j+1].

Simple correction

Swap the pair if the left value is larger.

Repeat passes

Large values “bubble” toward the right end after repeated scans.

[5, 1, 4, 2] → compare adjacent pairs left to right

Bubble Sort — pass 1

Array = [5, 1, 4, 2]

5 vs 1 swap	→	**[1, 5, 4, 2]**
5 vs 4 swap	→	**[1, 4, 5, 2]**
5 vs 2 swap	→	**[1, 4, 2, 5]**

After pass 1, the largest value 5 is already in its final position.

Bubble Sort — finish the trace

Continue with the unsorted prefix.

Pass 2 1 vs 4 → no swap 4 vs 2 → swap **[1, 2, 4, 5]**

Pass 3 1 vs 2 → no swap **[1, 2, 4, 5]**

Worst-case comparisons = $(n-1)+(n-2)+\cdots+1 = n(n-1)/2$

Basic Bubble Sort: $\Theta(n^2)$. With an early-stop flag, an already sorted array can finish in $\Theta(n)$.

Sequential Search: the simplest brute-force search

Inspect elements one by one until the key is found or the list ends.

Array = [18, 7, 24, 9, 31] key = 9

Comparison 1

18 ≠ 9

Comparison 2

7 ≠ 9

Comparison 3

24 ≠ 9

Comparison 4

9 = 9 ✓

The search succeeds after 4 comparisons.

Sequential Search — complexity

The cost depends on where the key is located.

Best case

Key is the first element.

1 comparison → Θ(1)

Worst case

Key is last or absent.

n comparisons → Θ(n)

Average successful case

If each position is equally likely:

about (n+1)/2 comparisons → Θ(n)

This is brute force because we use no ordering, index, hash table, or preprocessing — just inspect the data directly.

Exhaustive search: brute force over complete solutions

For optimization problems, the natural candidates may be whole tours, subsets, or assignments.

Generate

Enumerate every legal complete candidate solution.

Evaluate

Compute the cost, value, or feasibility of each candidate.

Choose

Keep the best valid candidate encountered.

The danger: candidate counts may grow as 2^n or $n!$

Exhaustive-search example: Traveling Salesman Problem

Visit every city once, return to the start, and minimize total distance.

	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Brute-force idea

Fix A as the start. Enumerate every ordering of B, C, and D; close the tour back to A; compute each total.

With 4 cities: (4−1)! = 3! = 6 tours from fixed start A

TSP — enumerate all six tours

Compute every candidate before choosing the minimum.

A-B-C-D-A	10+35+30+20	95
A-B-D-C-A	10+25+30+15	80
A-C-B-D-A	15+35+25+20	95
A-C-D-B-A	15+30+25+10	80
A-D-B-C-A	20+25+35+15	95
A-D-C-B-A	20+30+35+10	95

Minimum tour cost = 80. Two reverse-equivalent tours attain it in this symmetric example.

TSP — why brute force stops scaling

The exact candidate count grows factorially.

Fixed start city → approximately (n−1)! candidate tours

n=5

4! = 24

n=8

7! = 5,040

n=11

10! = 3,628,800

n=15

14! ≈ 87 billion

This is the classic warning sign for exhaustive search: even modest increases in n can multiply the work dramatically.

Exhaustive-search example: 0/1 Knapsack

Choose a subset of items whose total weight fits the capacity and whose total value is maximum.

Item	Weight	Value
A	2	6
B	3	10
C	4	12
D	5	15

Capacity = 7

For each item we have two choices: include it or exclude it.

With 4 items there are $2^4 = 16$ subsets.

Candidate = one subset of {A,B,C,D}

Knapsack — evaluate the feasible subsets

Discard overweight subsets; keep the feasible subset with maximum value.

∅

weight=0 value=0

A

weight=2 value=6

B

weight=3 value=10

C

weight=4 value=12

D

weight=5 value=15

AB

weight=5 value=16

AC

weight=6 value=18

AD

weight=7 value=21

BC

weight=7 value=22

Best feasible subset: {B,C} with weight 7 and value 22.

Knapsack — candidate explosion

Each item doubles the number of possible subsets.

n items → 2^n subsets

n=10

1,024

n=20

1,048,576

n=30

≈ 1.07 billion

n=40

≈ 1.10 trillion

Brute-force knapsack is exact, but exponential growth quickly motivates dynamic programming and other techniques later in the course.

Exhaustive-search example: Assignment Problem

Assign each worker to exactly one job while minimizing total cost.

	J1	J2	J3
W1	9	2	7
W2	6	4	3
W3	5	8	1

Candidate

A complete one-to-one assignment is a permutation of the jobs.

For 3 workers: 3! = 6 candidates.

General candidate count = n!

Assignment Problem — enumerate all 6 assignments

Write the job chosen by W1, W2, W3 in that order.

J1,J2,J3	9+4+1	**14**
J1,J3,J2	9+3+8	**20**
J2,J1,J3	2+6+1	**9**
J2,J3,J1	2+3+5	**10**
J3,J1,J2	7+6+8	**21**
J3,J2,J1	7+4+5	**16**

Best assignment: W1→J2, W2→J1, W3→J3 with total cost 9.

Assignment Problem — complexity

Permutations grow factorially just like TSP tour orderings.

n workers and n jobs → n! complete assignments

n=5	n=8	n=10	n=12
120	40,320	3,628,800	479,001,600

A direct exhaustive search is ideal for understanding the problem, but not for large instances.

Another textbook connection: DFS and BFS

Levitin also places depth-first and breadth-first traversal in the brute-force chapter.

Depth-First Search (DFS)

Systematically explores one path as deeply as possible before backtracking.

Uses a stack idea / recursion.

Breadth-First Search (BFS)

Systematically explores vertices level by level from the start.

Uses a queue.

We will study graph representations, DFS, BFS, connectivity, and traversal applications in the dedicated graph lecture.

What changes from one brute-force problem to another?

The philosophy is similar; the candidate space is what changes.

Problem	Candidate	How many?	Typical brute-force cost
Sequential search	array position	n	$\Theta(n)$
String matching	alignment	$n-m+1$	$O(nm)$
Inversion count	pair of indices	$n(n-1)/2$	$\Theta(n^2)$
Closest pair	pair of points	$n(n-1)/2$	$\Theta(n^2)$
Convex hull	candidate edge × point test	$\Theta(n^2) \times \Theta(n)$	$\Theta(n^3)$
Knapsack	subset	2^n	exponential
TSP / Assignment	permutation	$\approx n!$	factorial

The key analysis habit: count candidates, then multiply by the cost of testing one candidate.

Classroom discussion 1

If brute force can guarantee an optimal solution, why do we not always use it?

Think first

What happens to the number of candidates as n grows?

Would an exact answer still be useful if it takes years to compute?

Explanation

Optimality is valuable only when the computation is feasible. For large inputs, exhaustive search may require quadratic, exponential, or factorial work.

The practical goal is usually: correct enough + fast enough + within available memory.

Classroom discussion 2

What happens when the search space becomes extremely large?

$10^2 = 100$ | $2^{20} \approx 1{,}048{,}576$ | $20! \approx 2.43 \times 10^{18}$

Consequence

Runtime and sometimes memory become impractical long before every candidate can be checked.

What we try instead

Exploit structure, prune candidates, use dynamic programming, greedy methods, heuristics, approximation, or problem-specific algorithms.

Classroom discussion 3

Why might some problems be easier to optimize than others?

Helpful structure

Sorted order, monotonicity, overlapping subproblems, graph properties, or geometry may let us avoid unnecessary work.

Weak structure

If every candidate appears equally plausible, there may be little obvious work to skip.

Knowledge matters

A better representation or mathematical insight can reveal structure that was not obvious at first.

Optimization is often about proving that some work is unnecessary.

Classroom discussion 4

How does the structure of a problem influence our choice of algorithm?

Sorted data	→	**Binary Search instead of Linear Search**
Repeated overlapping subproblems	→	**Dynamic Programming**
Graph connectivity	→	**BFS / DFS**
Greedy-choice property	→	**Greedy algorithm**
Small unstructured candidate space	→	**Brute force may be perfectly reasonable**

Choose the algorithm after understanding the input, constraints, structure, and required output — not from the problem name alone.

Technique-selection lesson

Brute force is the baseline we compare other design techniques against.

Use brute force when

n is small, the task is infrequent, correctness and simplicity matter, or no better structure is available.

Improve it when

the same work repeats, the candidate space grows too quickly, or input structure can be exploited.

Keep it anyway as

a testing oracle, benchmark, fallback implementation, or conceptual reference solution.

A good algorithm designer should know both the simple baseline and the reason a more advanced technique improves it.

Final summary

Three problems, one design philosophy: test every natural candidate systematically.

Inversion count

pair of positions

$\Theta(n^2)$

Closest pair

pair of points

$\Theta(n^2)$

Convex hull

pair as edge + scan points

$\Theta(n^3)$

Candidate count × cost per candidate = total brute-force cost.

CSC301 • Design and Analysis of Algorithms
COMSATS University Islamabad, Attock Campus